Roteiro do vídeo de apresentação

Integrantes: Jander Luis Magalhães Coutinho e Arthur Fernandes Viana.

Duração planejada: aproximadamente 4min30s. Ensaiem com um cronômetro e mantenham o vídeo abaixo de 5 minutos. Comecem com o rosto visível e áudio. Se ambos participarem, Jander pode apresentar a arquitetura e Arthur demonstrar o programa e os testes.

As falas abaixo são uma sugestão para vocês explicarem o projeto com suas próprias palavras. Não é necessário mostrar todos os arquivos linha por linha.

Antes de gravar

Abra um terminal na pasta `cpu-simulada` e deixe estes comandos preparados:

```
py main.py --passo
py main.py --programa exemplo --resumo
py main.py --programa 8bits --resumo
py testes.py
```

No Linux/macOS, use `python3`; em instalações que disponibilizam `python`, ele também funciona. Aumente a fonte do terminal até os números ficarem legíveis no vídeo.

0:00 a 0:20 - Identificação, com rosto visível

Fala sugerida:

“Olá, somos Jander Luis Magalhães Coutinho e Arthur Fernandes Viana. Este é nosso trabalho de Arquitetura de Computadores: uma CPU simulada de oito bits, implementada em Python a partir do código-base da disciplina.”

0:20 a 1:00 - Componentes

Na tela: seção de arquitetura do README ou página inicial do guia.

Fala sugerida:

“A CPU possui quatro registradores, R0 a R3, e uma memória com dezesseis posições. Os registradores guardam os valores usados nas operações. A memória guarda as instruções e os dados. O PC indica o endereço da próxima instrução. A ULA faz soma, subtração e as operações lógicas AND, OR e XOR.”

“O ciclo funciona assim: primeiro busca a instrução usando o PC, depois identifica a operação e os operandos, e então executa. Geralmente o PC avança uma posição. Um salto muda o endereço, e HALT encerra a execução.”

1:00 a 1:25 - Organização do código e 8 bits

Na tela: arquivos `cpu.py`, `ula.py` e `programas.py`; mostre rapidamente a função `limitar_8bits`.

Fala sugerida:

“A classe CPU concentra os registradores, a memória e o controle da execução. A ULA fica em um arquivo separado, e os programas ficam em programas.py. Os valores são limitados a oito bits com uma máscara: valor AND 255. Assim, os dados armazenados ficam entre zero e 255.”

1:25 a 3:15 - Demonstração do programa próprio

Na tela: execute `py main.py --passo`. Pressione Enter para avançar e deixe o terminal acompanhar a saída.

Fala sugerida, acompanhando as instruções:

“Nosso programa possui treze instruções armazenadas. Vamos acompanhar os registradores antes e depois de cada instrução.”

“LOAD coloca 12 em R0 e 5 em R1. Aqui, LOAD recebe o valor diretamente, como no código-base. ADD soma os dois e grava 17 em R0. Em seguida, colocamos 3 em R2 e SUB reduz R0 para 14.”

“Agora usamos as operações lógicas, que trabalham bit a bit. AND entre 14 e 5 resulta em 4. OR entre 4 e 3 resulta em 7. XOR entre 7 e 5 resulta em 2. O AND mantém um bit ligado quando os dois bits são 1; o OR quando pelo menos um é 1; e o XOR quando são diferentes.”

“Na posição 8, JMP muda o PC para 10. Por isso, a instrução da posição 9, que colocaria 99 em R0, não é executada. STORE grava o resultado 2 na posição 15 e o valor 5 de R1 na posição 14. HALT encerra a execução.”

Depois de completar os 12 ciclos, mostre a tabela final e diga:

“A tabela mostra o PC e os registradores após cada instrução. Executamos doze ciclos porque uma instrução foi pulada. Ao final, R0 vale 2, R1 vale 5, R2 vale 3 e R3 continua em zero. O PC fica em 12, na instrução HALT.”

3:15 a 3:40 - Programa do enunciado

Na tela: execute `py main.py --programa exemplo --resumo`.

Fala sugerida:

“Também executamos o exemplo do enunciado. Ele carrega 5 e 8, soma e grava o resultado na memória. Como esperado, a posição 15 contém 13.”

3:40 a 4:05 - Limites de 8 bits

Na tela: execute `py main.py --programa 8bits --resumo` e aponte as linhas ADD e SUB.

Fala sugerida:

“Esta demonstração verifica os limites. Quando somamos 255 e 1, o resultado armazenado vira zero porque só mantemos oito bits. Depois, zero menos um gera o padrão binário 11111111, mostrado como 255 na representação sem sinal.”

4:05 a 4:30 - Testes e encerramento

Na tela: execute `py testes.py` e mostre `Ran 36 tests` e `OK`.

Fala sugerida:

“Os 36 testes automáticos passaram. Eles verificam as operações, transferências, avanço e salto do PC, parada, programas completos e limites de oito bits. A simulação permite observar como cada instrução muda o estado da CPU. Obrigado.”

Respostas para perguntas prováveis

O que é uma CPU simulada? Um programa que representa componentes e executa instruções de uma CPU pequena. Não simula todos os detalhes de um processador real.

Qual a diferença entre registrador e memória? O registrador guarda os valores usados diretamente pela CPU. A memória contém o programa e os dados armazenados.

O que é o PC? É o contador de programa, que aponta para a próxima instrução. Não é um dos quatro registradores gerais.

O primeiro ou o segundo registrador recebe a soma? O primeiro. Em `ADD R0, R1`, o resultado vai para R0. R1 é apenas lido.

LOAD lê uma posição da memória? Nesta implementação, não. `LOAD R0, 5` coloca o próprio número 5 em R0, seguindo a tabela de instruções e o código-base.

Por que 14 AND 5 dá 4? 14 é `00001110` e 5 é `00000101`. Só o bit que vale 4 é 1 nos dois números.

Por que 7 XOR 5 dá 2? `00000111` e `00000101` diferem apenas no bit que vale 2.

Por que há 13 instruções, mas 12 ciclos? JMP pula o endereço 9. Ele nunca aparece entre os PCs executados.

Por que o PC fica em 12 no final? HALT interrompe a execução sem incrementar o PC, como no código-base.

Por que os dados ficam nas posições 14 e 15? As instruções ocupam 0 a 12. Essas duas posições estão livres e podem receber dados sem sobrescrever o programa.

A memória inteira é composta por bytes reais? Não. É uma lista Python de 16 posições; instruções são textos e dados são números limitados a 8 bits. É a representação didática adotada.

O que vocês completaram no código-base? STORE, as cinco operações da ULA e sua integração à CPU. Também acrescentamos os programas, testes, validações e visualização antes/depois com modo passo a passo.

O que acontece em um loop infinito? A execução é interrompida após o limite de ciclos, por padrão 100, com uma mensagem de erro. Isso é uma proteção do simulador.

Entrega

O enunciado exige um PDF no SIGAA com os nomes e os links do repositório e do vídeo. Prazo: **21/10/2026 às 23h55**. O link do repositório já está no PDF preparado. Depois de gravar e disponibilizar o vídeo, inserir o link do vídeo no campo indicado e salvar o documento.